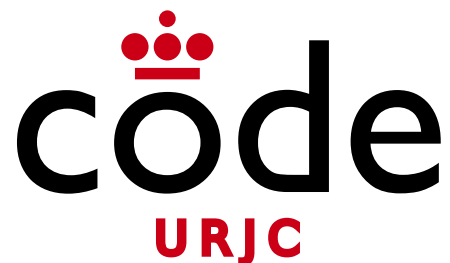


Fundamentos de la Web

Bloque III: Tecnologías de servidor web

Tema 3.2: Bases de datos



Fundamentos de la Web

Bloque III: Tecnologías de servidor web

Tema 3.2.3: Base de datos MongoDB en Node

Índice

- MongoDB en Node
- Inserción de documentos
- Consulta de documentos
- Actualización de documentos
- Borrado de documentos
- Gestión de objetos en arrays
- Conclusiones

Índice

- **MongoDB en Node**
- Inserción de documentos
- Consulta de documentos
- Actualización de documentos
- Borrado de documentos
- Gestión de objetos en arrays
- Conclusiones

MongoDB en Node

- MongoDB tiene un **cliente oficial** para Node
- Permite invocar operaciones de forma **similar** a la shell de Mongo (pero hay diferencias)
- Instalar

```
$ npm install --save mongodb
```

<https://www.mongodb.com/docs/drivers/node/current/>

MongoDB en Node

ejem1-insert-query

```
import { MongoClient } from 'mongodb';

const client = new MongoClient('mongodb://localhost:27017');

const db = client.db('cinema');
const movies = db.collection('movies');

await movies.insertOne({
  title: "Back to the Future",
  year: 1985
});

const movie = await movies.findOne({ title: "Back to the Future" });

console.log(movie);

client.close();
```

MongoDB en Node

insert-query

Cliente de la
base de datos

```
import { MongoClient } from 'mongodb';

const client = new MongoClient('mongodb://localhost:27017');

const db = client.db('cinema');
const movies = db.collection('movies');

await movies.insertOne({
  title: "Back to the Future",
  year: 1985
});

const movie = await movies.findOne({ title: "Back to the Future" });

console.log(movie);

client.close();
```

MongoDB en Node

ejem1-insert-query

```
import { MongoClient } from 'mongodb';

const client = new MongoClient('mongodb://localhost:27017');

const db = client.db('cinema');
const movies = db.collection('movies');

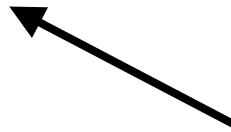
await movies.insertOne({
  title: "Back to the Future",
  year: 1985
});

const movie = await movies.findOne({ title: "Back to the Future" });

console.log(movie);

client.close();
```

IP y puerto de la
base de datos



MongoDB en Node

ejem1-insert-query

```
import { MongoClient } from 'mongodb';

const client = new MongoClient('mongodb://localhost:27017');

const db = client.db('cinema');
const movies = db.collection('movies');

await movies.insertOne({
  title: "Back to the Future",
  year: 1985
});

const movie = await movies.findOne({ title: "Back to the Future" });

console.log(movie);

client.close();
```

Acceso a la db
y colección



MongoDB en Node

ejem1-insert-query

```
import { MongoClient } from 'mongodb';

const client = new MongoClient('mongodb://localhost:27017');

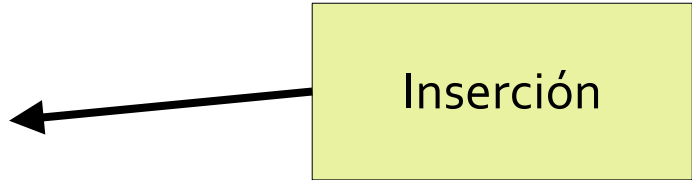
const db = client.db('cinema');
const movies = db.collection('movies');

await movies.insertOne({
  title: "Back to the Future",
  year: 1985
});

const movie = await movies.findOne({ title: "Back to the Future" });

console.log(movie);

client.close();
```



MongoDB en Node

ejem1-insert-query

```
import { MongoClient } from 'mongodb';

const client = new MongoClient('mongodb://localhost:27017');

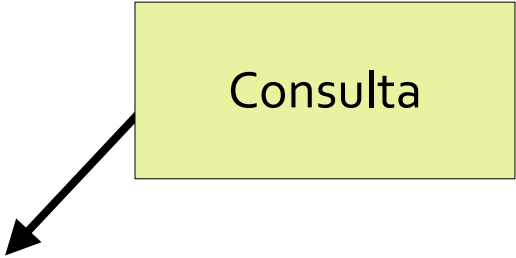
const db = client.db('cinema');
const movies = db.collection('movies');

await movies.insertOne({
  title: "Back to the Future",
  year: 1985
});

const movie = await movies.findOne({ title: "Back to the Future" });

console.log(movie);

client.close();
```



Async / Await

- En JavaScript, una función que se conecta a la **base de datos (entrada/salida)** se ejecuta de forma **asíncrona**
- A las funciones asíncronas se las suele invocar anteponiendo **await**

```
await movies.insertOne({
  title: "Back to the Future",
  year: 1985
});
```

```
const movie = await movies.findOne({ title: "Back to the Future" });
```

Async / Await

ejem2-aync-await

- Si en la implementación de una función se usa **await**, la función debe declararse **async** (asíncrona)
- Una función **async** suele invocarse con **await**

```
async function insertDemoMovie() {
  await movies.insertOne({
    title: "Back to the Future",
    year: 1985
  });
}

await insertDemoMovie();
```

Más adelante se profundizará en **async / await**

Índice

- MongoDB en Node
- **Inserción de documentos**
- Consulta de documentos
- Actualización de documentos
- Borrado de documentos
- Gestión de objetos en arrays
- Conclusiones

Insertar un documento

ejem3-insert-id

- Añadir un documento:

```
await movies.insertOne({
  title: "Back to the Future",
  year: 1985
});
```

- `insertOne(...)` devuelve el `id` generado

```
let result = await movies.insertOne({
  title: "Back to the Future",
  year: 1985
});

console.log(`Inserted id: ${result.insertedId}`);
```

Insertar un documento

- `insertOne(...)` añade el `id` al objeto

ejem3-insert-id

```
let movie = {
  title: "The Goonies",
  year: 1985
}

await movies.insertOne(movie);

console.log(`Inserted id: ${movie._id}`);
```


Insertar varios documentos

ejem3-insert/memory

- Insetar varios documentos:

```
await customers.insertMany([
  { firstName: 'Jack', lastName: 'Bauer' },
  { firstName: 'Juan', lastName: 'Pérez' }
]);
```

- Obtener los ids

```
const result = await customers.insertMany([
  { firstName: 'Jack', lastName: 'Bauer' },
  { firstName: 'Juan', lastName: 'Pérez' }
]);

console.log('Customers inserted with ids:', result.insertedIds);
// Customers inserted with ids: {
//   '0': 5df95777099a07467dbe941d,
//   '1': 5df95777099a07467dbe941e
// }
```

Insertar varios documentos

ejem3-insert/file

- Insetar varios cargados de un fichero:

```
import { readFile } from 'fs/promises';

...

// Get absolute path from file name
let dataFileName = 'customers.json';
let currentSourceFilePath = import.meta.url;
let absoluteDataFilePath = new URL(dataFileName, currentSourceFilePath);

// Read file contents as String
const customersAsString = await readFile(absoluteDataFilePath, 'utf8');

// Convert string to objects in memory
const customersAsObjects = JSON.parse(customersAsString);

// Insert into database
await customers.insertMany(customersAsObjects);
```

Índice

- MongoDB en Node
- Inserción de documentos
- **Consulta de documentos**
- Actualización de documentos
- Borrado de documentos
- Gestión de objetos en arrays
- Conclusiones

Consultar documentos

ejem4-query

- Consultar un documento por su id
- Consultar varios documentos
- Consultar por valor de atributos (filtros)
- Consultas ordenadas
- Consultas paginadas
- Consultas con proyección
- Consultas con buscador de texto
- Consultas con cálculos

Consultar documentos

- Consultar un documento por su id ejem4-query/findOne

```
import { MongoClient, ObjectId } from 'mongodb';

...

let id = ...

let customer = await customers.findOne({ _id: new ObjectId(id) });
```

Consultar documentos

ejem4-query/findCursor

- Consultar de varios documentos

- Procesar documentos uno a uno (cursor)

```
const resultCursor = customers.find();

for await (const doc of resultCursor) {
  console.log(doc);
}
```

- Cargar todos los documentos en memoria

```
const resultArray = await customers.find().toArray();

console.log(resultArray);
```

Consultar documentos

ejem4-query/findQuery

- Consultar por valor de atributos (filtros)
 - Consultar todos

```
const result = customers.find();
```

<https://www.mongodb.com/docs/drivers/node/current/crud/query/retrieve/#finding-documents>

- Consultar con un valor de un atributo

```
const result = customers.find({ age: 15 });
```

- Consultar con una expresión (menor que)

```
const result = customers.find({ age: { $lt: 18 } });
```

<https://www.mongodb.com/docs/drivers/node/current/crud/query/query-document/#comparison-operators>

Consultar documentos

ejem4-query/findQuery

- Consultar por valor de atributos (filtros)
 - Consultar con dos atributos diferentes AND

```
const result = customers.find({ age: { $gte: 18 }, country: 'Spain' });
```

<https://www.mongodb.com/docs/drivers/node/current/crud/query/query-document/#logical-operators>

- Consultar con dos atributos iguales AND

```
const result = customers.find({ $and: [{age: {$gte: 18}}, {age: {$lte: 20}}]});
```

- Consultar con varios valores para un atributo

```
const result = customers.find({ country: { $in: ['Spain', 'England']} });
```


Consultar documentos

ejem4-query/findQuery

- Consultar por valor de atributos (filtros)
 - Consultar con dos atributos OR

```
const result = customers.find({ $or: [{ gender: 'female' }, { country: 'Spain' }]});
```

- Consultar con una expresión regular

```
const result = customers.find({ name: { $regex: "Pérez" } });
```

Consultar documentos

ejem4-query/findSort

- Consultas ordenadas
 - Se indica el campo de ordenación

```
const result = customers.find().sort({ name: 1 });
```

Consultar documentos

ejem4-query/findPaginated

- Consultas paginadas
 - Se puede especificar los elementos de una página con numPage y pageSize

```
let resultsPage = await customers.find()
    .skip((numPage - 1) * pageSize)
    .limit(pageSize);
```

Consultar documentos

ejem4-query/findProjected

- Consultas con proyección
 - Se puede especificar las propiedades del documento que se obtienen en la consulta
 - Por defecto se incluye el `_id` a no ser que se indique lo contrario

```
const result = customers.find().project({ _id: 0, name: 1});
```

Consultar documentos

ejem4-query/findQuery

- Consultas con buscador de texto

```
let searchBar = "ramo";
const result = customers
  .find({ $text: { $search: searchBar } })
  .sort({ score: { $meta: "textScore" } });
```

- Es necesario configurar la colección para búsquedas de texto

```
const db = ...
const customers = db.collection('customers');

//Create index if not already created
await customers.createIndex({ name: 'text' });
```

Consultar documentos

- Consultas con cálculos
 - Agrupar valores de múltiples documentos
 - Realizar operaciones para obtener un único resultado (sumas, medias, conteo)
 - Analizar cómo los datos cambian a lo largo del tiempo

Aggregation Operations

Ejercicio 1

ejer1-movies-template

- Cargar los datos del fichero `movies.json` en una base de datos Mongo y muestra la siguiente información:
 - El nombre y el año de las películas de los años 80 (1980 – 1989)
 - Toda la información de las películas de Star Wars
 - Nombre y año de las 5 películas más recientes
 - El nombre y género de las películas románticas o que tengan “love” en su título (con regex)

Índice

- MongoDB en Node
- Inserción de documentos
- Consulta de documentos
- **Actualización de documentos**
- Borrado de documentos
- Gestión de objetos en arrays
- Conclusiones

Actualizar documentos

ejem5-update

- Actualizar un campo de un documento
- Reemplazar un documento

Actualizar documentos

ejem5-update/updateOne

- Actualizar un campo de un documento

```
let id = ...;

let updateResult = await customers.updateOne(
  { _id: new ObjectId(id) }, { $set: { age: 40 } });

if(updateResult.matchedCount == 0){
  console.log(`No document found with id ${id}`);
} else {
  if(updateResult.modifiedCount == 0){
    console.log(`No document modified`);
  } else {
    console.log(`Document updated`);
  }
}
```

Actualizar documentos

ejem5-update/replaceOne

- Reemplazar un documento

```
let id = ...

let newCustomer = {
  name: "Pepito Perezito",
  age: 37,
  country: 'Spanistan'
}

let replaceResult = await customers.replaceOne(
  { _id: new ObjectId(id) }, newCustomer);

if(replaceResult.matchedCount == 0){
  console.log(`No document found with id ${id}`);
} else {
  if(replaceResult.modifiedCount == 0){
    console.log(`No document modified`);
  } else {
    console.log(`Document updated`);
  }
}
```

Índice

- MongoDB en Node
- Inserción de documentos
- Consulta de documentos
- Actualización de documentos
- **Borrado de documentos**
- Gestión de objetos en arrays
- Conclusiones

Borrar documentos

- Borrar un documento
- Borrar varios documentos

ejem6-delete

Borrar documentos

ejem6-delete/deleteOne

- Borrar un documento

```
let id = ...

let deleteResult = await customers.deleteOne(
  { _id: new ObjectId(id) });

if(deleteResult.deletedCount == 0){
  console.log(`No document found with id ${id}`);
} else {
  console.log(`Document deleted`);
}
```

Borrar documentos

ejem6-delete/deleteMany

- Borrar varios documentos

```
const deleteResult = await customers.deleteMany();  
console.log(`Deleted documents: ${deleteResult.deletedCount}`);
```

Índice

- MongoDB en Node
- Inserción de documentos
- Consulta de documentos
- Actualización de documentos
- Borrado de documentos
- **Gestión de objetos en arrays**
- Conclusiones

Gestionar objetos en arrays

ejem7-arrays

- Mongo permite que las **propiedades** de los **documentos** sean de tipo **array**
- Ese array puede tener **objetos**

```
{
  name: "Juan",
  tasks: [
    { title: "Learn JavaScript", done: true },
    { title: "Learn MongoDB", done: false },
    { title: "Learn Express", done: false }
  ]
}
```

Gestionar objetos en arrays

ejem7-arrays

- Para poder manipular cada **objeto de forma independiente**, es necesario que cada uno tenga un **id**

```
{
  _id: ObjectId('68fb8643baccc0c3e7024c50')
  name: "Juan",
  tasks: [
    { _id: ObjectId('68fb8643baccc0c3e7024c51'), title: "Learn JavaScript", done: true },
    { _id: ObjectId('68fb8643baccc0c3e7024c52'), title: "Learn MongoDB", done: false },
    { _id: ObjectId('68fb8643baccc0c3e7024c53'), title: "Learn Express", done: false }
  ]
}
```

Gestionar objetos en arrays

ejem7-arrays

- Por defecto Mongo **no genera** el id para los objetos del array, hay que especificar un atributo `_id`

```
_id: new ObjectId(),
```

```
const newUser = {
  name: "Juan",
  tasks: [
    { _id: new ObjectId(), title: "Learn JavaScript", done: true },
    { _id: new ObjectId(), title: "Learn MongoDB", done: false },
    { _id: new ObjectId(), title: "Learn Express", done: false }
  ]
}

let result = await users.insertOne(newUser);
```

Gestionar objetos en arrays

ejem7-arrays

- Insertar un nuevo objeto en el array

```
let userId = ...
let newTask = { _id: new ObjectId(), title: "Learn...", done: false }

await users.updateOne(
  { _id: new ObjectId(userId) },
  { $push: { tasks: newTask } }
);

console.log(`Task inserted with id ${newTask._id}`);
```

Gestionar objetos en arrays

ejem7-arrays

- Modificar un objeto del array
 - Poner done: true

```
let userId = ...
let taskId = ...

await users.updateOne(
  { _id: new ObjectId(userId), "tasks._id": new ObjectId(mongoTaskId)},
  { $set: { "tasks.$.done": true } }
);
```

Gestionar objetos en arrays

ejem7-arrays

- Reemplazar un objeto del array

```
let userId = ...
let taskId = ...

let newTask = { _id: taskId, title: "Learn Pascal", done: true };

await users.updateOne(
  { _id: new ObjectId(userId), "tasks._id": new ObjectId(taskId) },
  { $set: { "tasks.$": newTask } }
);
```

Gestionar objetos en arrays

ejem7-arrays

- Cargar un objeto del array

```
let userId = ...
let taskId = ...

const userWithTask = await users.findOne(
  { _id: new ObjectId(userId) },
  { projection: { tasks: { $elemMatch: { _id: new ObjectId(taskId) } } } }
);

let task = userWithTask.tasks[0];
```

Gestionar objetos en arrays

ejem7-arrays

- Borrar un objeto del array

```
let taskId = ...
let userId = ...

await users.updateOne(
  { _id: new ObjectId(userId) },
  { $pull: { tasks: { _id: new ObjectId(taskId) } } }
);
```


Índice

- MongoDB en Node
- Inserción de documentos
- Consulta de documentos
- Actualización de documentos
- Borrado de documentos
- Gestión de objetos en arrays
- **Conclusiones**

Conclusiones

ejem7-arrays

- Mongo ofrece un cliente para **insertar, consultar, actualizar, reemplazar y borrar** documentos de forma persistente (en disco)
- La conversión entre un documento es **directa en JavaScript**
- Los documentos pueden tener **objetos y arrays**, lo que permite representar **información estructurada**